Duplicate Question Detection

Quora Question Pairs: Project Report and Recommendations

Prepared by: Priyashree, AI Engineer Trainee, Rubixe

Executive Summary

The goal was to build a model that predicts whether two questions have the same meaning, using the Quora Question Pairs dataset of **404,290** labelled pairs. Three approaches were built and compared on the same held-out set of 80,858 pairs.

Model	Accuracy	Precision	Recall	F1
Engineered features + Logistic Regression	66.93%	54.15%	68.05%	60.31%
TF-IDF similarity + engineered features	67.62%	54.88%	69.23%	61.22%
Sentence Transformer + engineered features	**78.42%**	**65.94%**	**85.93%**	**74.62%**

> **Headline:** Adding semantic embeddings (`all-MiniLM-L6-v2`) lifted F1 by 14.3 points and recall by 17.9 points over the baseline. The model finds about 86% of true duplicates, but roughly 1 in 3 of its duplicate flags is wrong, so it is best used to **suggest** duplicates for review rather than to auto-merge questions.

1. Business Case and Goal

The task is to predict which pairs of questions carry the same meaning. Labels come from human experts and are inherently subjective and noisy, so they represent a reasonable consensus rather than absolute truth. Model performance should be read with that ceiling in mind.

Project goal: create a model that predicts which question pairs are duplicates.

2. Data Understanding

Item	Finding
Size	404,290 question pairs, 6 columns (id, qid1, qid2, question1, question2, is_duplicate)
Class balance	255,027 non-duplicates (63.08%) and 149,263 duplicates (36.92%), moderately imbalanced
Missing values	1 in question1 and 2 in question2; the affected rows were dropped, leaving 404,287 pairs
Exact duplicate rows	None
Unique questions	537,929 unique questions across the pairs, so many questions appear in more than one pair
Average length	About 59.5 characters (10.9 words) for Q1 and 60.1 characters (11.2 words) for Q2

What separates duplicates from non-duplicates

- **Length:** duplicate pairs are shorter (about 52.9 characters, 9.9 words) than non-duplicates (about 63.5 to 64.3 characters, 11.6 to 12.0 words).
- **Word overlap (Jaccard):** 0.431 for duplicates vs 0.264 for non-duplicates.
- **Fuzzy string similarity** (10,000-pair sample): mean 70.1 for duplicates vs 56.3 for non-duplicates.
- Both signals help, but the distributions overlap heavily, so surface-level text similarity alone cannot separate the classes well.

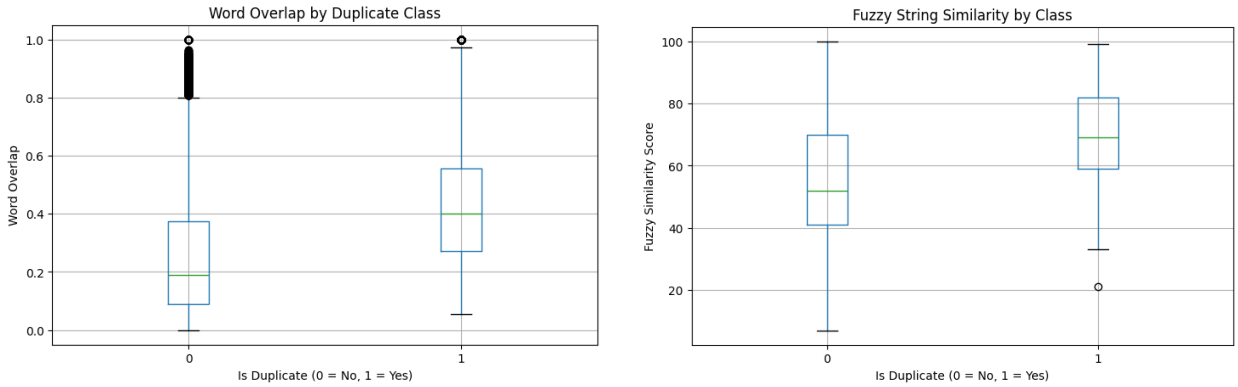


Figure 1. Word overlap (left) and fuzzy similarity (right) by class

3. Work Completed

The work was done in a structured sequence, each step building on the last:

Step	What was done
1. Data audit	Loaded the dataset, checked shape, types, class balance, missing values and exact duplicates; reviewed sample pairs from both classes.
2. Cleaning	Dropped the 3 rows with a missing question; confirmed no duplicate rows remain.
3. Feature engineering	Built seven features: question lengths (chars), word counts, word overlap ratio, fuzzy ratio and fuzzy partial ratio (fuzzywuzzy).
4. Data split	Stratified 80/20 split with a fixed seed: 323,429 training and 80,858 test pairs, with identical class ratios (63.08 / 36.92).
5. Early exploration	Tried TF-IDF (unigrams + bigrams, 100K features) on concatenated question text with Logistic Regression, and tested the sentence transformer on a 5,000-pair sample.
6. Model 1: baseline	Logistic Regression on the 7 engineered features (class-weight balanced).
7. Model 2: TF-IDF	Cosine similarity between the TF-IDF vectors of each question, added to the 7 features (8 features total).
8. Model 3: semantic	Encoded all questions with `all-MiniLM-L6-v2` (384-dimensional embeddings) on a Tesla T4 GPU, computed cosine similarity per pair, and added it to the 7 features.
9. Evaluation	Compared all models on the same test set using accuracy, precision, recall, F1 and confusion matrices.
10. Error analysis	Inspected false positives and false negatives, and studied similarity distributions by class.
11. Threshold study	Swept decision thresholds from 0.30 to 0.70 to show the precision/recall trade-off.

4. Results

All three models use Logistic Regression with balanced class weights and are evaluated on the same 80,858-pair test set.

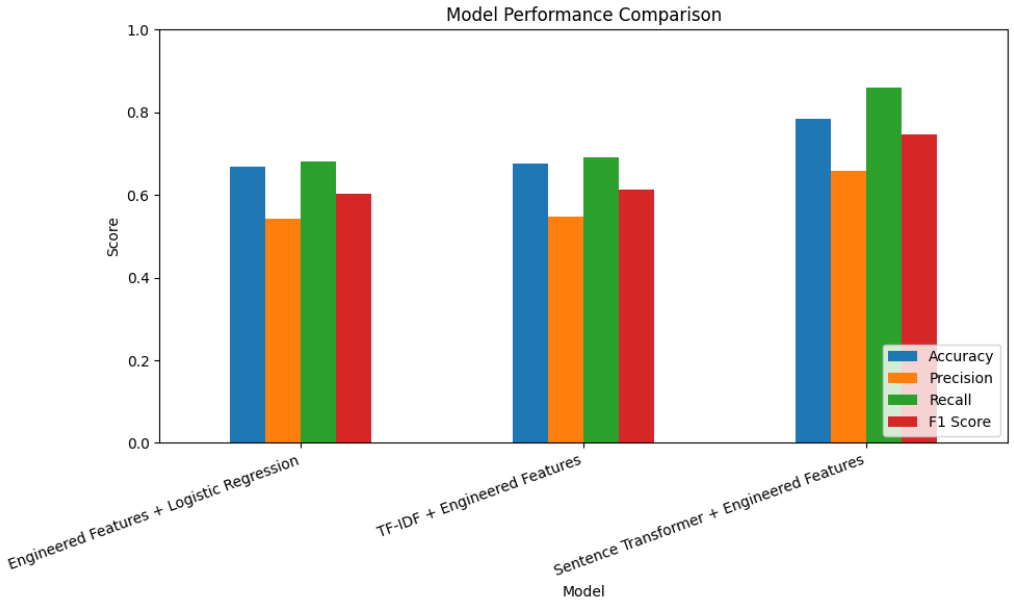


Figure 2. Performance comparison across the three approaches

Comparison vs baseline	Accuracy	Precision	Recall	F1
Sentence Transformer model	+11.48 pts	+11.78 pts	+17.88 pts	+14.31 pts

Confusion matrices (test set, 80,858 pairs)

Model	True neg.	False pos.	False neg.	True pos.
Baseline (engineered features)	33,806	17,199	9,537	20,316
TF-IDF + features	34,013	16,992	9,186	20,667
Sentence Transformer + features	37,752	13,253	4,199	25,654

The semantic model cut missed duplicates (false negatives) by more than half, from 9,537 to 4,199, and also reduced false positives by about 3,900. The TF-IDF step added only about 0.7 points of accuracy, showing that word-level similarity carries little beyond what the basic features already capture.

5. Error Analysis

The final model made 17,452 errors on the test set: **13,253 false positives** and **4,199 false negatives**. False positives dominate, which matches the model's high-recall, lower-precision profile.

Why false positives happen

Pairs that are on the same topic but ask for different things get high similarity scores. Examples from the test set:

Question 1	Question 2	Issue
What is the scope after doing M.Tech. from IIT?	What are the opportunities after doing M.Tech from good IITs?	Near-paraphrase that was labelled non-duplicate
Which phone should I buy Moto Z Play or OnePlus 3T?	Should I buy OnePlus 3T or go for any other phone of the same price range?	Overlapping entities, different question
How can I make a billion dollars?	What are some ways to make 1,000 dollars?	Same topic, very different quantity
How can I get Android Nougat?	What is Android Nougat?	Same subject, different intent

Why false negatives happen

True duplicates that are phrased very differently, or where one question is much longer or contains extra context, get low scores. For example, *"Which are the best books to learn C++?"* was paired with a long question about learning C++ for a specific IIT project. Very short vs very long paraphrases are a known weakness of comparing two independent embeddings.

Similarity distributions

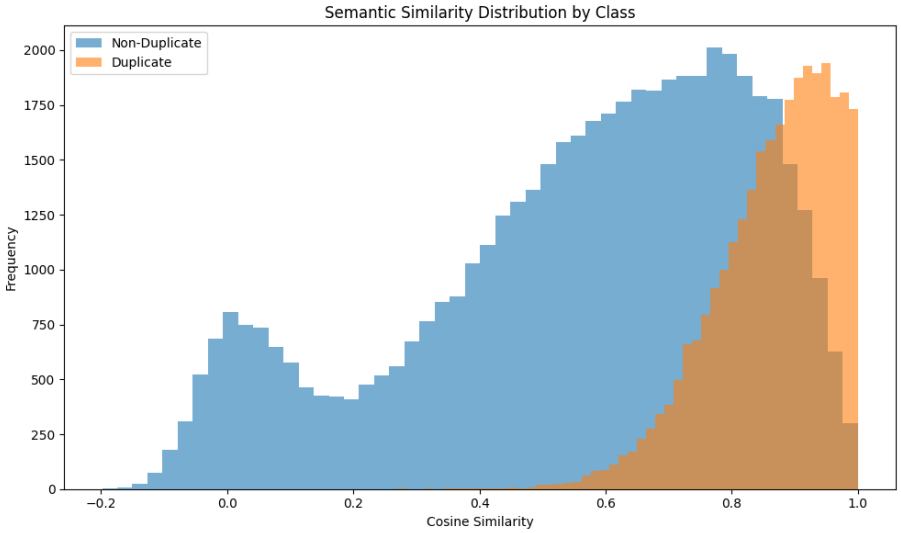


Figure 3. Semantic similarity by class. Duplicates cluster high (mean 0.865, std 0.095); non-duplicates are spread widely (mean 0.561, std 0.271).

Note that many non-duplicates also score above 0.8. This overlap is the root cause of the false positives and explains why one similarity feature cannot go much beyond the mid-70s F1.

6. Decision Threshold Trade-off

The default cut-off of 0.50 maximises F1, but the right threshold depends on how the client will use the model. The sweep below shows the trade-off:

Threshold	Accuracy	Precision	Recall	F1
0.30	73.99%	59.24%	94.72%	72.89%
0.35	75.44%	60.97%	92.99%	73.65%
0.40	76.60%	62.58%	91.04%	74.18%
0.45	77.57%	64.22%	88.63%	74.48%
0.50 (default)	**78.42%**	**65.94%**	**85.93%**	**74.62%**
0.55	78.90%	67.55%	82.45%	74.26%
0.60	79.23%	69.39%	78.29%	73.57%
0.65	79.25%	71.38%	73.10%	72.23%
0.70	78.94%	73.62%	66.96%	70.13%

- **Lower thresholds (0.30 to 0.40):** catch 91 to 95% of duplicates. Suits a system that suggests related questions or flags candidates for review.
- **Around 0.50:** best balance of precision and recall (highest F1).
- **Higher thresholds (0.65 to 0.70):** precision climbs to 71 to 74%, but recall falls to 67 to 73%. Suits cases where a wrong merge is costly.

7. Advice to the Client

> **Bottom line:** Use the Sentence Transformer model now as a **duplicate-suggestion engine with human review**, and invest in the improvements below before allowing any fully automatic merging of questions.

A. How to use the current model

- **Adopt the semantic model.** It is clearly better than lexical approaches and is cheap to run: small model, simple classifier, no heavy infrastructure.
- **Keep a human in the loop.** With about 66% precision at the default threshold, roughly one in three flagged pairs is not a real duplicate. Auto-merging would wrongly combine distinct questions.
- **Choose the threshold by business goal.** Use about 0.35 to 0.40 to surface candidates for reviewers, about 0.50 for balanced screening, and 0.65 or higher if wrong merges are costly.
- **Plan for label noise.** The labels are subjective, so some "errors" are really label disagreements. Treat accuracy in the high 70s as a starting point, not a defect.

B. Improvements, in priority order

Priority	Improvement	Why it matters
1	**Cross-encoder** (e.g. a BERT-style model that reads both questions together)	Directly attacks the main failure: topically similar questions with different intent. Typically the biggest accuracy gain for this task.
2	**Fine-tune the sentence transformer** on the labelled Quora pairs	Teaches the embedding what "duplicate" means for this data instead of generic similarity.
3	**Hard-negative mining**	Train on non-duplicates with very high similarity (the false-positive type shown above) so the model learns the difference.
4	**Intent, entity and number features**	Catches cases like "a billion dollars" vs "1,000 dollars", or "how to get X" vs "what is X".
5	**Stronger classifier** (XGBoost/LightGBM) and probability calibration	Lets the model use richer features, and makes probabilities trustworthy for threshold setting.
6	**Deploy as an API / web app**	Two questions in; duplicate flag plus similarity score out. Lets reviewers give feedback that becomes new training data.

C. Validation practices to adopt before production

- **Split by question, not only by pair.** The dataset has 537,929 unique questions in 404,287 pairs, so the same question can appear in both train and test, which can make scores look better than real-world performance. A grouped split would give a more honest estimate.
- **Tune the threshold on a separate validation set.** The threshold sweep was run on the test set; for a final, unbiased number, hold out a third set.
- **Use multiple splits or cross-validation** to report a performance range rather than a single figure.
- **Monitor after launch.** Track precision from reviewer decisions and retrain periodically as topics drift.

8. Limitations and Notes

- The dataset labels are noisy and subjective; a perfect score is not achievable.
- Results are from a single 80/20 split with a fixed seed, so they are a point estimate, not a confidence range.
- An early exploratory run (TF-IDF over the concatenated text of both questions with Logistic Regression) reached 78.34% accuracy, 72.14% F1 and 0.863 ROC-AUC. That is close to the final model's accuracy, but this approach does not measure similarity between the two questions directly, so it was set aside for the structured comparison above. It is worth retesting as an extra feature source.
- Transformer experiments on 5,000-pair samples were used only for quick validation; the headline numbers all come from the full 80,858-pair test set.
- The sentence model was used off-the-shelf without fine-tuning.

9. Conclusion

The project built and compared three duplicate-detection approaches on 404K Quora question pairs. Semantic similarity from `all-MiniLM-L6-v2` combined with engineered features gave the best result: **78.42% accuracy, 85.93% recall and 74.62% F1**, which is +14.31 F1 points over the baseline. The remaining errors are mostly topically-similar questions with different intent, which points clearly to cross-encoders, fine-tuning and hard-negative mining as the next steps.

I recommend deploying the current model as a reviewer-assist tool with an adjustable threshold, and funding the cross-encoder and fine-tuning work as the next phase.

Appendix: Tools and Setup

Area	Details
Language / libraries	Python, pandas, NumPy, scikit-learn, fuzzywuzzy, sentence-transformers, PyTorch, matplotlib, seaborn
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (384-dimensional)
Classifier	Logistic Regression (max_iter=1000, class_weight=balanced, random_state=42)
Hardware	Google Colab, Tesla T4 GPU
Split	80/20 stratified, random_state=42